



BIG DATA · CIÊNCIA DA COMPUTAÇÃO · UFLA

Pipeline de Stream Processing sobre Dados de Táxi de Nova York

*Apache Kafka + Spark Structured Streaming
de um host único a um cluster distribuído na AWS*

GRUPO g11

NYC TLC Trip Record Data · Docker · Kafka · Spark · AWS

Contexto e Motivação



Corridas de táxi e apps geram um fluxo contínuo de eventos de mobilidade — não um arquivo estático.



Perguntas operacionais (picos de demanda, tempo médio de viagem) perdem valor se respondidas em lote, horas depois.



Proposta: consumir os eventos quase em tempo real com Kafka + Spark Structured Streaming.



Pergunta experimental central do projeto: adicionar máquinas realmente acelera o pipeline?

Por que é Big Data



Volume

Subconjunto experimental: $\approx 1,07$ GB
comprimido / $\approx 48,8$ milhões de linhas
— não cabe confortavelmente na RAM.



Velocidade

Fluxo contínuo de alta vazão publicado
no Kafka — tratado como firehose, não
como arquivo estático.



Shuffle

Agregações stateful redistribuem dados
pela rede entre nós — o gargalo
clássico, cujo custo cresce com volume
e nº de nós.

Gargalo central do projeto: o custo do shuffle nas agregações stateful — é isso que os experimentos medem.

Os Dados — NYC TLC Trip Record Data

Dados públicos e reais da New York City Taxi & Limousine Commission, com quatro serviços de schemas heterogêneos.

Serviço	Modalidade	Coluna event-time	Campo de tarifa
yellow	Táxi amarelo (medallion)	tpep_pickup_datetime	fare_amount
green	Táxi verde (boro taxi)	lpep_pickup_datetime	fare_amount
fhv	For-hire vehicle	pickup_datetime	(ausente)
fhvhv	High-volume FHV (Uber/Lyft)	pickup_datetime	base_passenger_fare

≈ 2,3 GB / 102M

linhas baixadas — 4 meses, 4 serviços

≈ 1,07 GB / 48,8M

linhas — subconjunto de 2 meses usado nos experimentos

566 KB / 21 mil

corridas — amostra incluída p/ quick start

Por que é Big Data: o subconjunto de 1,07 GB não cabe confortavelmente na RAM, forçando processamento particionado.

Arquitetura do Pipeline

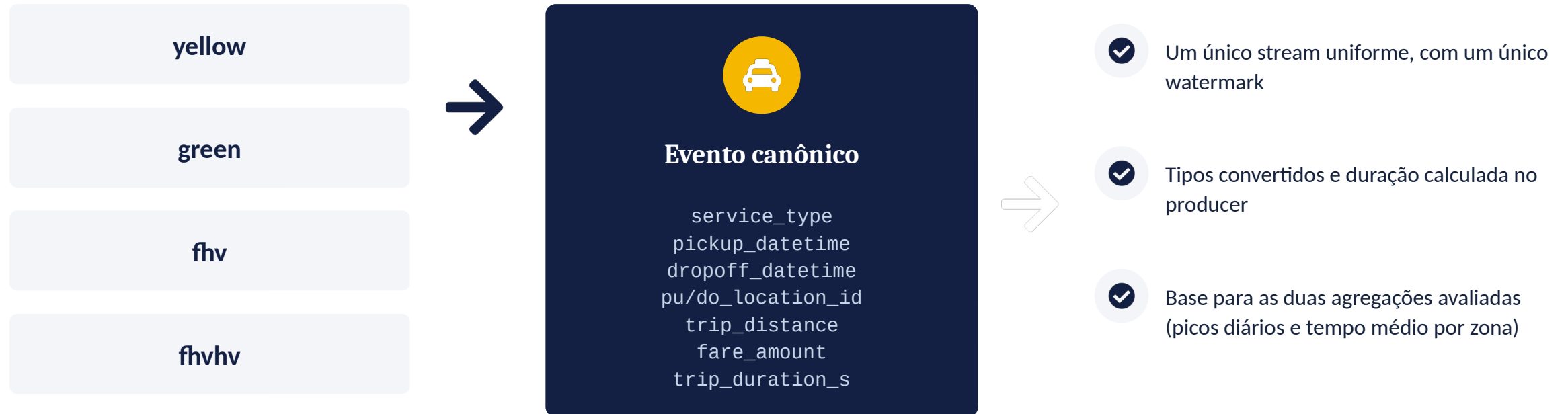


Fluxo linear único; as duas fases experimentais (host único e cluster AWS) usam o mesmo pipeline lógico, mudando apenas o substrato (Compose vs. Swarm).

Componente	Contêiner / arquivo	Papel
Ingestão	producer	lê Parquet, normaliza, publica no Kafka
Mensageria	kafka (KRaft)	firehose pub/sub, buffer durável
Processamento	spark-master + spark-worker	consome, filtra, agrega (shuffle), escreve
Saída	output/	resultados Parquet + checkpoints

O Evento Canônico

A chave da solução: unificar 4 schemas heterogêneos em um único evento, com um único watermark.



Workloads Avaliados

WORKLOAD-1

Picos diários

daily_peaks

Agregação stateful com watermark:

```
groupBy(window(pickup_datetime,  
"1 day")).count()
```

Número de corridas por janela diária. Provoca shuffle por chave de janela.

WORKLOAD-2

Tempo médio por zona

avg_duration_by_zone

Agregação + join com tabela de zonas:

```
groupBy(window(1 dia),  
pu_location_id).avg(duration)  
+ JOIN zonas
```

Combina shuffle de agregação e de join — a carga mais cara avaliada.

Metodologia Experimental



Throughput

registros/s processados pelo Spark



Latência

de micro-batch (ms), via
StreamingQueryListener



3 repetições

por configuração — média $\pm$ desvio
padrão

CENÁRIO A

Escala vertical

`spark.executor.cores` $\in \{1, 2, 4\}$ — nº de workers
fixo

CENÁRIO B

Escala horizontal

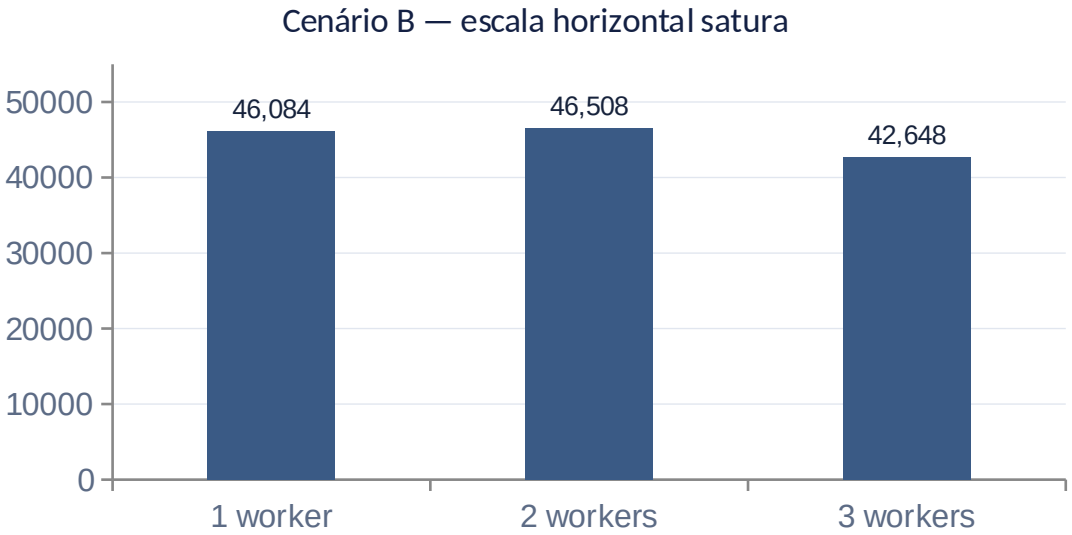
nº de workers $\in \{1, 2, 3\}$ — cores fixo em 4

Fase 1 — host único: 12 núcleos, 16 GB RAM, Docker Compose. **Fase 2 — cluster AWS:** 4 VMs EC2, Docker Swarm + rede overlay.

Host Único (12 núcleos)

3 repetições por configuração • 2M mensagens/run

Cenário	Config	Throughput (reg/s)	Latência (ms)
A (cores)	1 core	23 424,5	42 701,8
A (cores)	2 cores	32 544,6	30 731,3
A (cores)	4 cores	46 333,6	21 582,7
B (workers)	1 worker	46 084,2	21 712,0
B (workers)	2 workers	46 508,3	21 503,3
B (workers)	3 workers	42 648,4	23 447,8

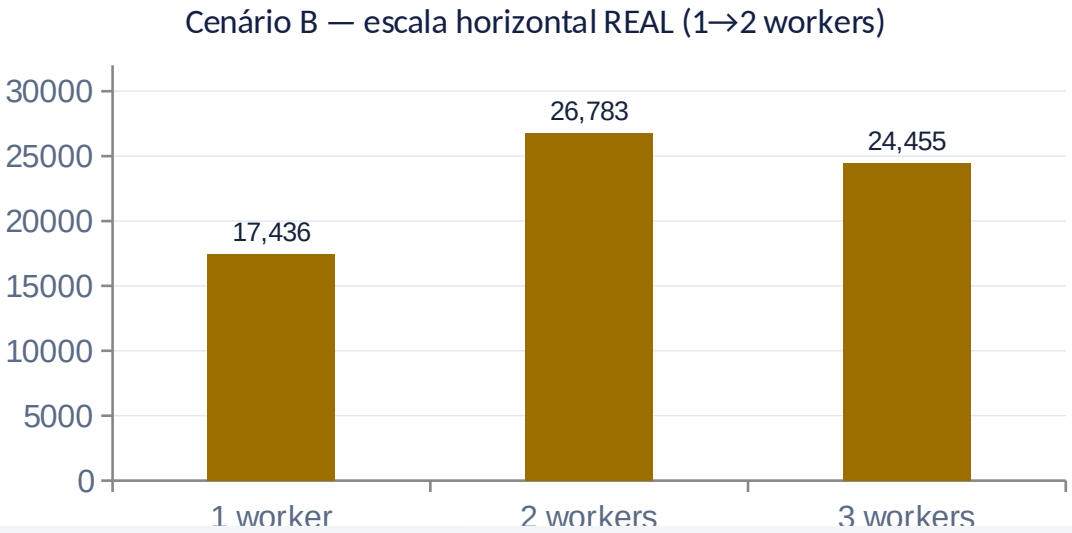


Escala vertical funciona: 1→4 cores ≈ +98% de throughput. **Escala horizontal satura:** 1→3 workers não acelera (piora em 3, por overhead num só host).

Cluster AWS (4 VMs • Docker Swarm)

3 repetições por configuração • 1 worker = 1 VM física (max_replicas_per_node: 1)

Cenário	Config	Throughput (reg/s)	Latência (ms)
A (cores)	1 core	19 965,5	50 167,3
A (cores)	2 cores	24 915,4	40 138,7
B (workers)	1 worker	17 435,6	59 712,2
B (workers)	2 workers	26 783,1	37 410,0
B (workers)	3 workers	24 454,5	40 896,5



No cluster AWS, com cada worker numa VM física distinta, a escala horizontal realmente escala: **+54% de throughput de 1→2 workers**. Satura em 3 workers — teto de um cluster pequeno de instâncias burstable.

Escala horizontal só entrega ganho quando há máquinas físicas distintas

+54%

throughput ao dobrar de 1 → 2 workers
no cluster AWS (17 435 → 26 783 reg/s)

HOST ÚNICO



1→3 workers NÃO acelera
(satura / piora em 3)

CLUSTER AWS



1→2 workers ESCALA
(+54% throughput real)

Por quê: no host único, workers concorrem pelos mesmos 12 núcleos físicos — não há paralelismo real adicional. No cluster AWS, cada worker roda numa VM distinta, então mais workers trazem mais CPU física de verdade.

Conclusões e Limitações



O que funcionou

- Pipeline stream (Kafka + Spark) roda out-of-the-box só com Docker
- Ingere e agrega os dados com sucesso nas duas fases experimentais
- Desenho experimental demonstra o ponto central: escala horizontal só entrega ganho com máquinas físicas distintas



Limitações

- Host único simula "horizontal" com processos concorrentes
- Cluster AWS pequeno (4 VMs) com instâncias t3 burstable
- Chave de particionamento `service_type` tem só 4 valores, concentrando o shuffle
- Subconjunto de $\approx 48,8$ M linhas, não os ≈ 102 M completos
- Latência medida é de micro-batch, não end-to-end por evento

Trabalhos Futuros



Cluster maior, CPU dedicada

Evitar o throttling de instâncias burstable e testar com mais nós físicos.



Rebalancear a chave de partição

Ir além de service_type (4 valores) para reduzir a concentração do shuffle.



Semântica exactly-once

Garantir processamento sem duplicidade nem perda de eventos ponta a ponta.



Latência end-to-end

Medir do evento de origem até o resultado, não só o micro-batch.



Obrigado

Perguntas?

Pipeline de Stream Processing sobre NYC TLC Trip Record Data · Big Data · UFLA · Grupo g11